

PCB Defect Detection

Project in Machine Learning Operations
ZHAW · Spring 2026

Project Goal

Automate PCB quality control using a self-reinforcing MLOps pipeline.

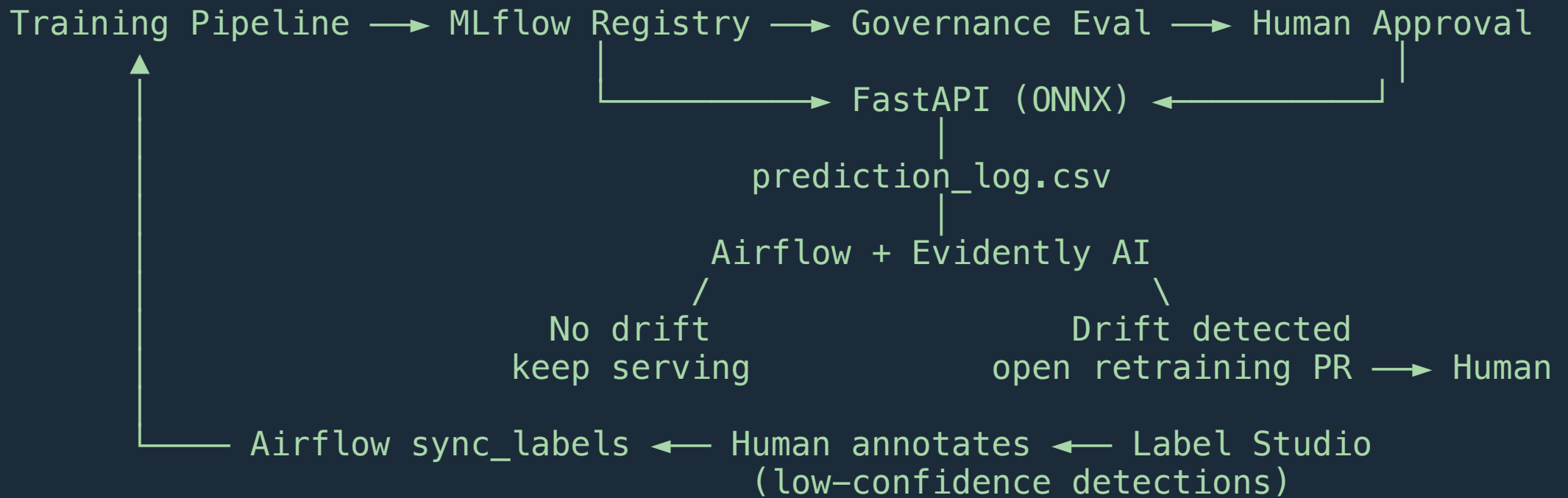
- Detect **6 defect types** on printed circuit boards:
`open` · `short` · `mousebite` · `spur` · `spurious_copper` · `pin_hole`
- Build a **flywheel** — every production run generates data that improves the next model
- Full **human-in-the-loop** governance: no model reaches production without human approval

Why this matters

Manual PCB inspection is slow, expensive, and error-prone.

An automated, self-improving system reduces defect escape rates over time.

System Architecture



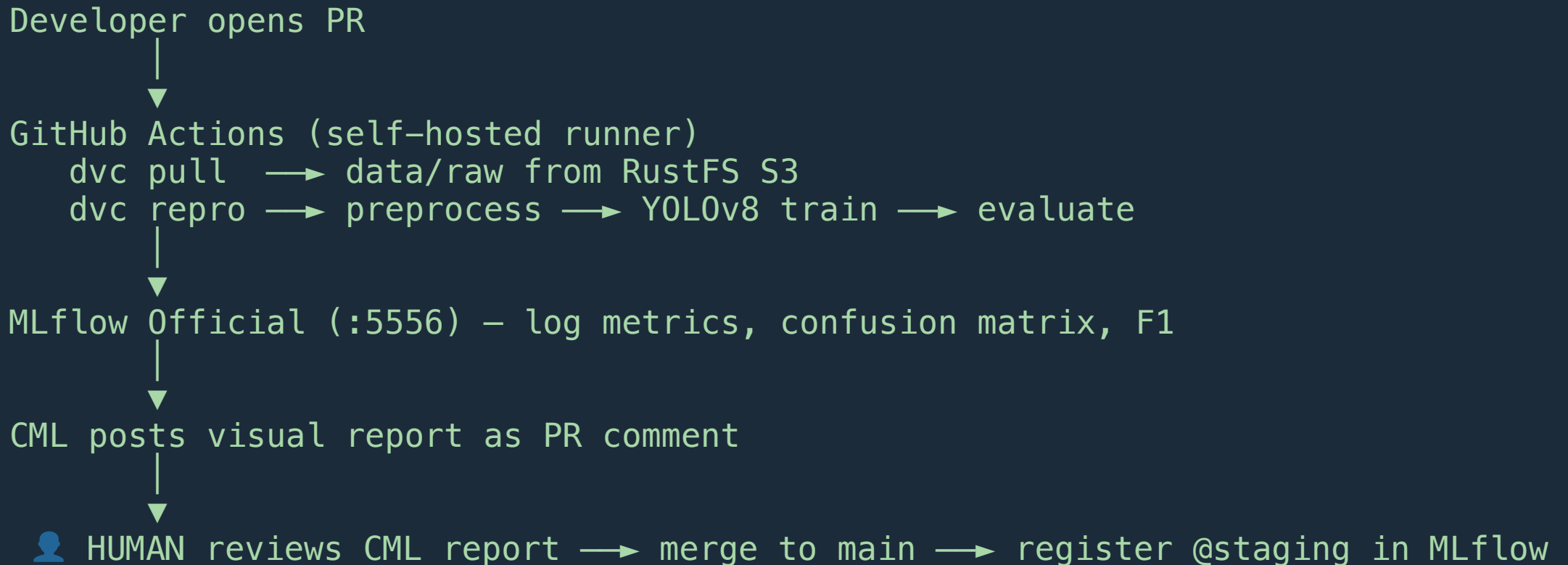
Gold gates = human decisions · Dashed = automated flywheel loops

Tech Stack

Component	Technology	Role
Code & CI/CD	Git · GitHub · GitHub Actions + Self-hosted Runner	Version control, automated training on PRs
Detection	YOLOv8 (ONNX export)	Object detection — 6 defect classes
Data Versioning	DVC + RustFS (S3)	Reproducible datasets, local S3-compatible storage
Experiment Tracking	MLflow (dual instance)	Sandbox :5555 for dev · Official :5556 for CI/CD
Workflow Orchestration	Airflow	Governance eval · label sync · drift monitoring
Drift Detection	Evidently AI	Statistical drift on confidence, bbox area, pass/fail
Annotation	Label Studio	Human labelling of low-confidence

Phase 1 — Training Pipeline

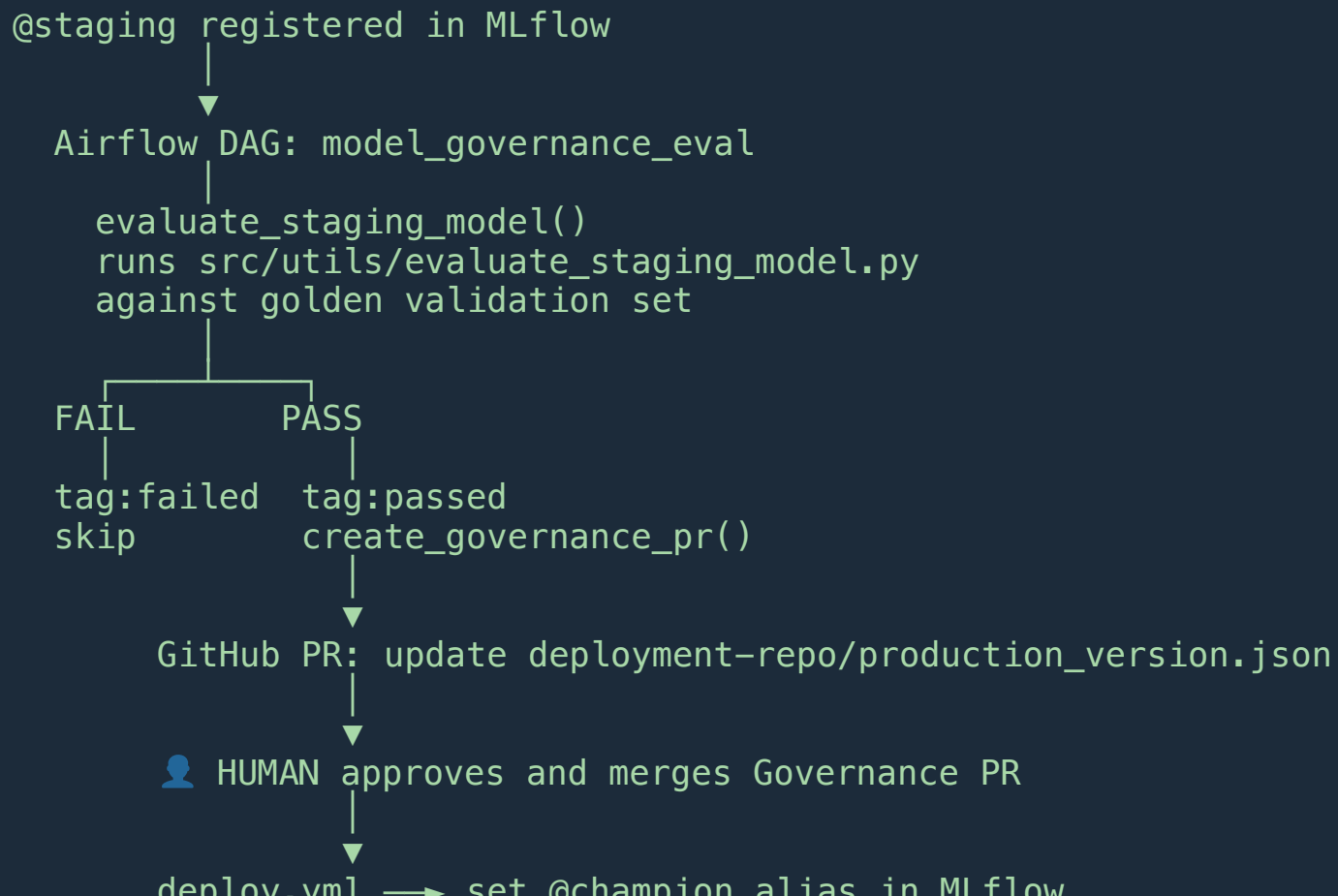
Triggered by: opening or updating a Pull Request



- **Fast iteration:** `uv run python src/training/train.py` → logs to Sandbox

Phase 2 — Model Governance

Triggered by: CI/CD push to `main` (after merge)



Phase 3 — Production Serving & Drift Monitoring

FastAPI loads `@champion` from MLflow · polls every 30 s for new version

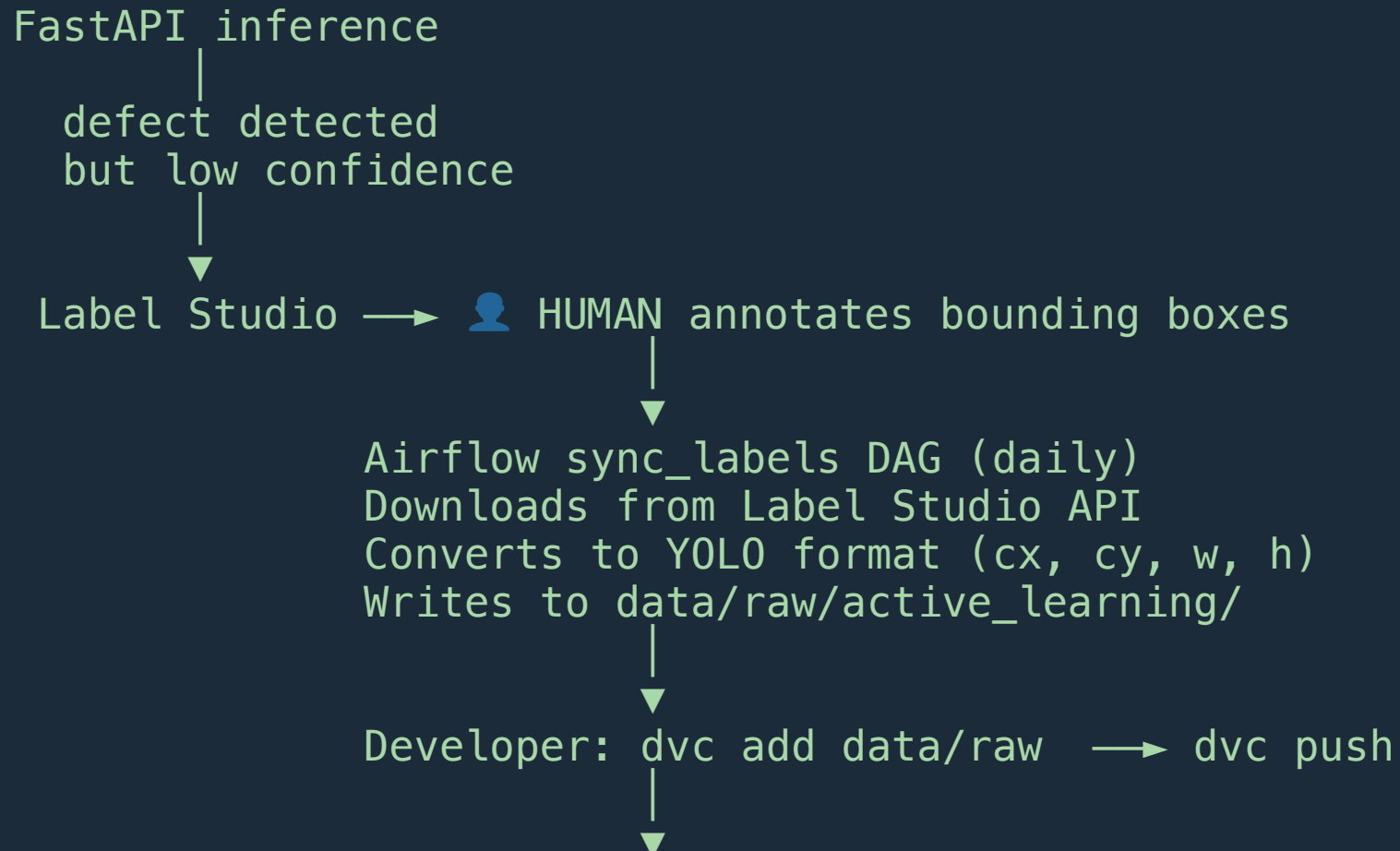
```
POST /predict → YOLOv8 ONNX inference
                  |
                  prediction_logger.log()
                  |
                  monitoring/prediction_log.csv
```

Airflow `data_sync_and_drift_check` **DAG** — runs daily:

Step	What it does
① <code>sync_labels</code>	Pull completed annotations from Label Studio → <code>data/raw/active_learning/</code>
② <code>drift_monitor</code>	Read last 100 rows of <code>prediction_log.csv</code> · run Evidently AI

Phase 4 — Active Learning

Closing the flywheel: uncertain predictions become tomorrow's training data



CI/CD Workflow

Self-hosted runner is required — it runs on the same machine as Docker, giving the pipeline direct access to MLflow, RustFS, and Airflow.

Pull Request opened / updated



ci.yml → dvc pull (from RustFS)
→ dvc repro (preprocess + YOLOv8 + eval)
→ log to MLflow Official (:5556)
→ CML: post confusion matrix + F1 curves to PR

Merge to main



ci.yml → export best.pt → best.onnx
→ register @staging in MLflow Model Registry
→ curl Airflow to trigger model_governance_eval DAG

Governance PR merged

Human-in-the-Loop Gates

Four checkpoints where a human must approve before the system advances:

Gate	Where	Decision
H1 — Annotation	Label Studio	Draw bounding boxes on low-confidence PCB images
H2 — Training PR	GitHub PR + CML report	Accept or reject model by reviewing confusion matrix and F1 vs main
H3 — Governance PR	GitHub Governance PR	Final safety gate before model reaches production — approving promotes <code>@champion</code>
H4 — Retraining PR	GitHub Drift PR	Decide whether detected drift warrants a retrain cycle

No model reaches production without a human merge.

No retraining happens without a human decision.

Demo

Streamlit UI · <http://localhost:8501>

Upload a PCB image → get defect bounding boxes + PASS / FAIL

FastAPI Docs · <http://localhost:8000/docs>

Airflow · <http://localhost:8085>

MLflow Official · <http://localhost:5556>

Evidently AI · <http://localhost:8005>

Label Studio · <http://localhost:8080>

-
- **Flywheel design:** drift → annotation → retrain → better model → less drift
 - **Three Airflow DAGs** orchestrate all automation: governance, sync+drift, deploy
 - **Four human gates** ensure no model reaches production unchecked
 - **ONNX format** eliminates Mac vs Linux inference discrepancies
 - **Dual MLflow** keeps sandbox experiments separate from official CI/CD runs
 - **Self-hosted runner** bridges GitHub Actions with local Docker infrastructure

PCB Defect Detection MLOps Flywheel · ZHAW Spring 2026